OPENHACK HACKATHON
Nginx Web Server — Complete Technical Documentation

Participant: MATHAN S | Kali Linux | 08 October 2026

This document records the implementation, verification procedure, troubleshooting, and terminal evidence for all five organizer tasks. Exactly five screenshots are included under each task, for a total of 25 screenshots.

Task	Requirement	Main Evidence	Final Verification
1	Nginx + custom HTML	Nginx status, web root, HTML	curl HTTP
2	Access from another computer	IP, SSH, HTTP response	Second-machine HTTP
3	Remote modification	SSH/HTML workflow	Changed HTTP response
4	Different page on port 8002	Second site config	curl :8002
5	HTTPS	OpenSSL + Nginx SSL config	curl -k HTTPS

Task 1 — Set Up Nginx and Serve the Custom HTML Page

Install/configure Nginx on the local Kali Linux machine and use it to serve the organizer-provided custom HTML page.

- Server IP: `10.10.145.207`
- Web root: `/var/www/html`
- Web server: Nginx
- Default HTTP port: `80`

The terminal evidence shows Nginx being located and its systemd service being checked.

which nginx

sudo systemctl status nginx

The service output shows the Nginx service running.

cd /var/www/html

ls

cat index.html

The existing default page was inspected before replacing it with the organizer's page.

The supplied custom HTML was copied into `/var/www/html/index.html`.

The screenshots also document permission problems encountered while changing files under `/var/www/html`. Administrative operations were therefore performed with `sudo`.

sudo nginx -t

Expected successful output:

nginx: the configuration file /etc/nginx/nginx.conf syntax is ok

nginx: configuration file /etc/nginx/nginx.conf test is successful

sudo systemctl restart nginx

Nginx was successfully configured and running. The custom HTML page was placed in the Nginx document root and subsequently returned by HTTP requests.

Five screenshots are included in `evidence/task-1-nginx/`.

Show:

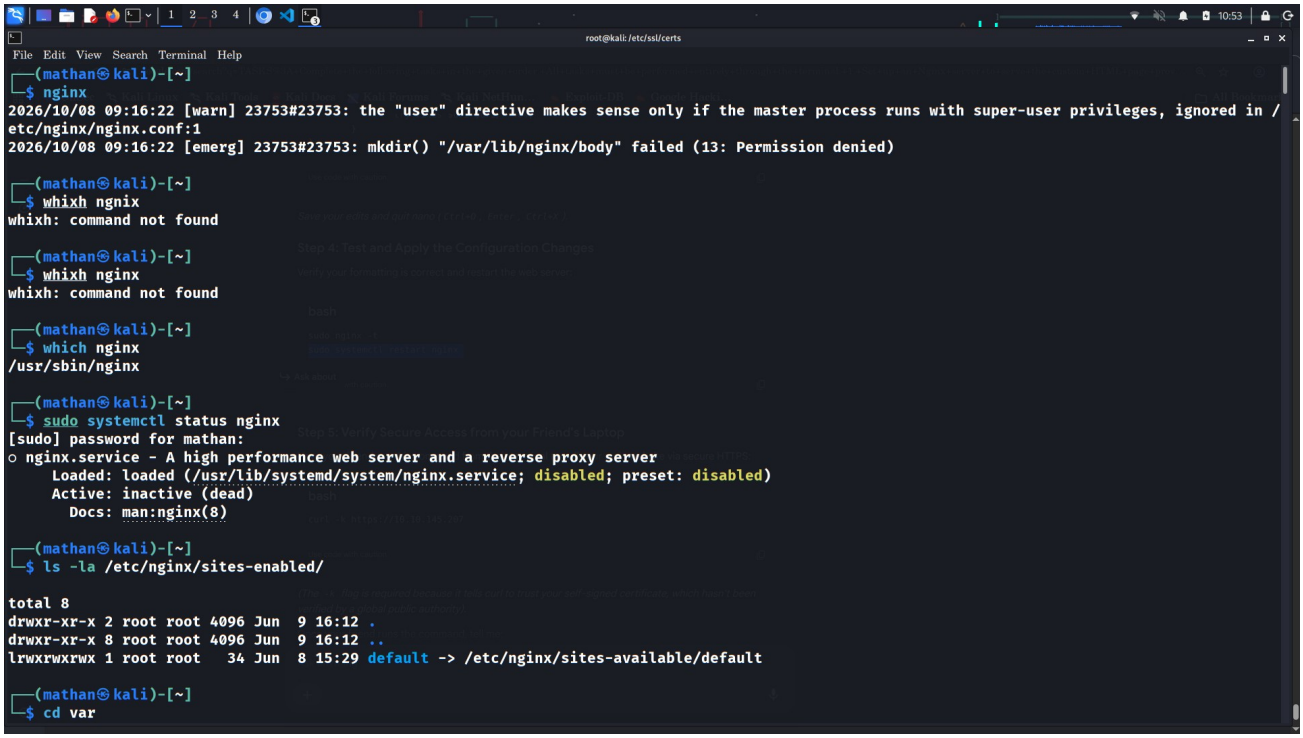
sudo systemctl status nginx

curl http://10.10.145.207

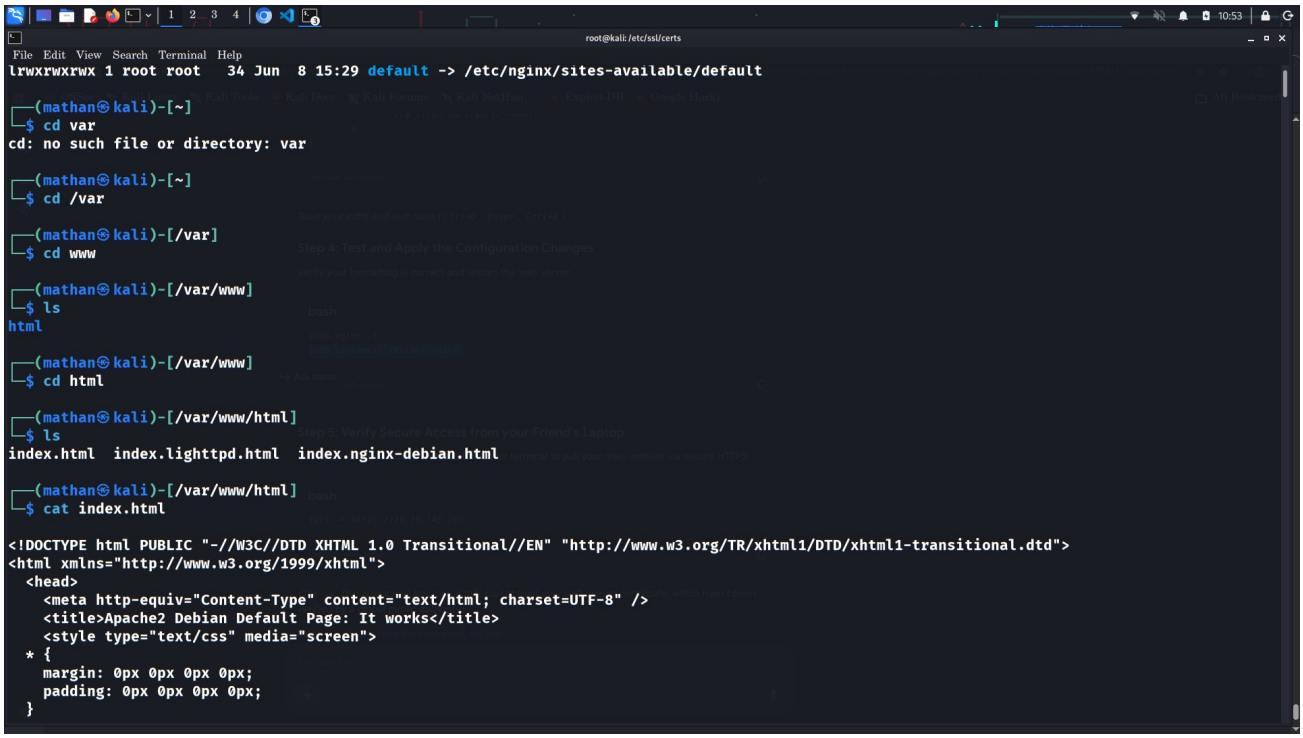
Then show the returned custom HTML page.

Terminal Evidence — 5 Screenshots

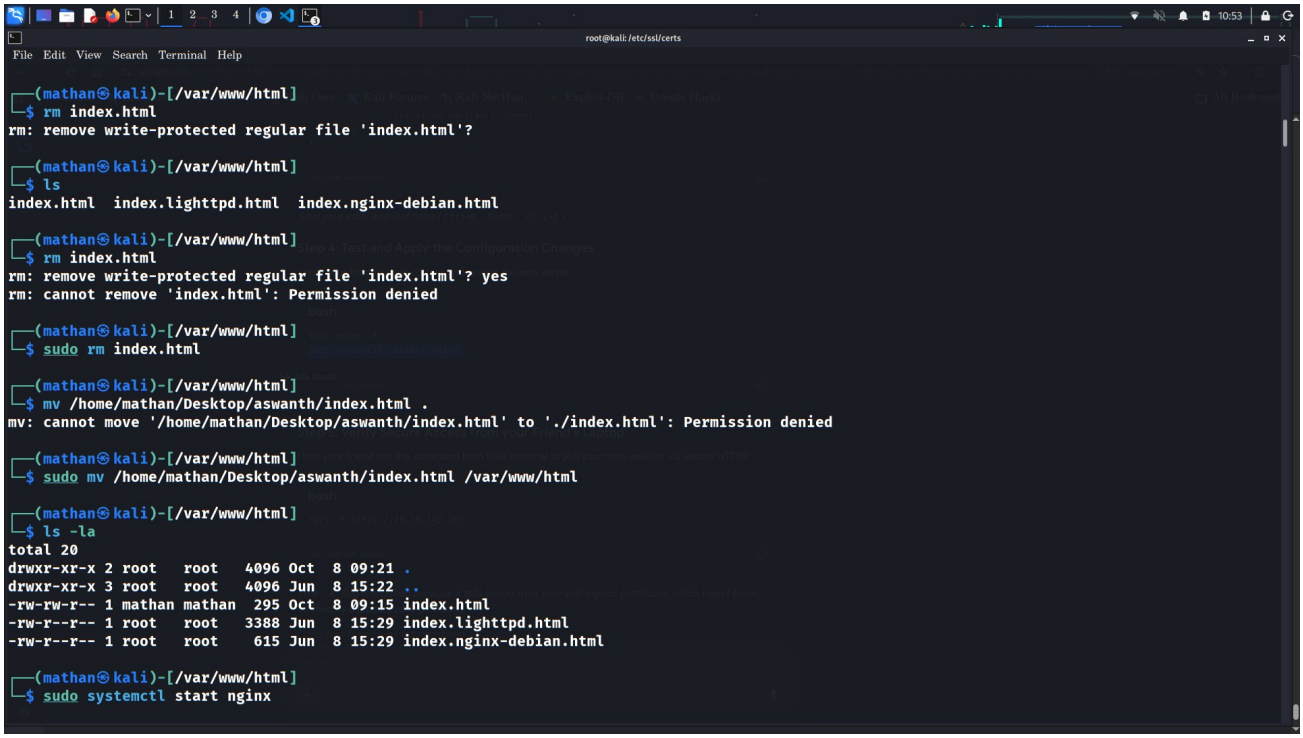
Evidence 1/5 — Screenshot_2026-10-08_10_53_31.png



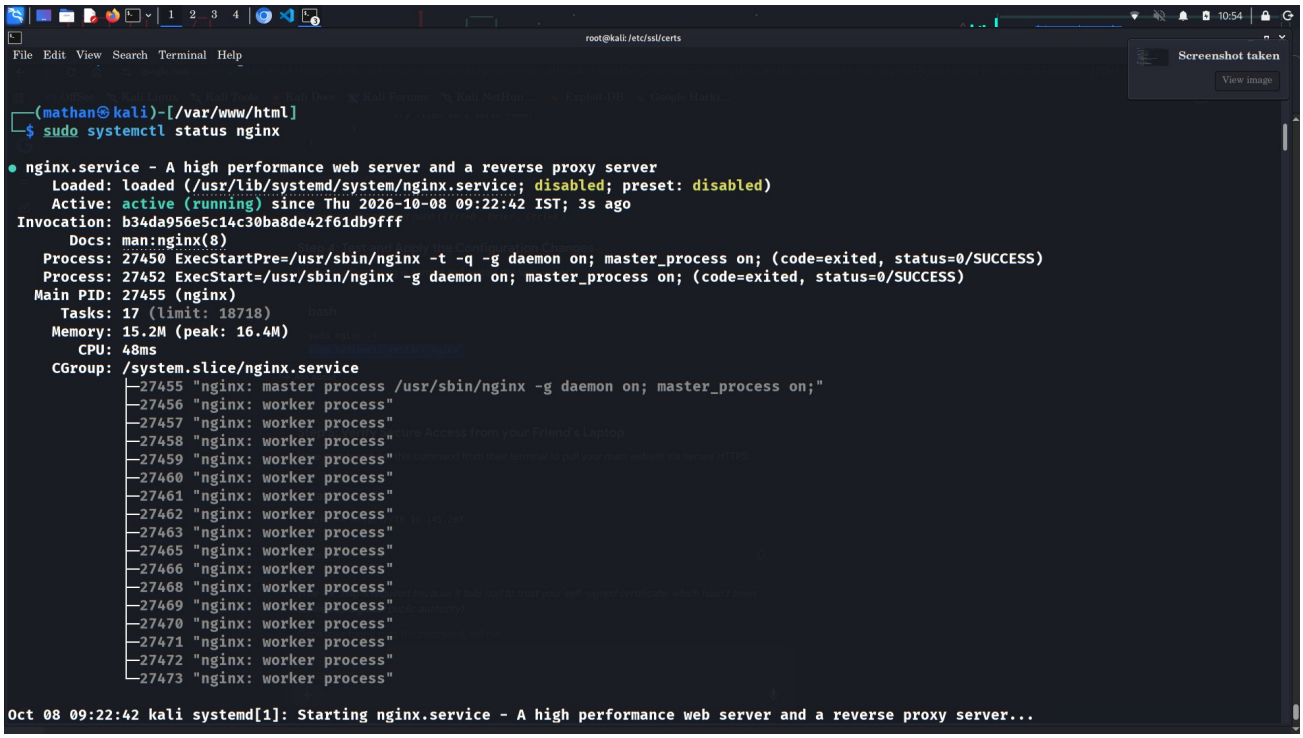
Evidence 2/5 — Screenshot_2026-10-08_10_53_46.png



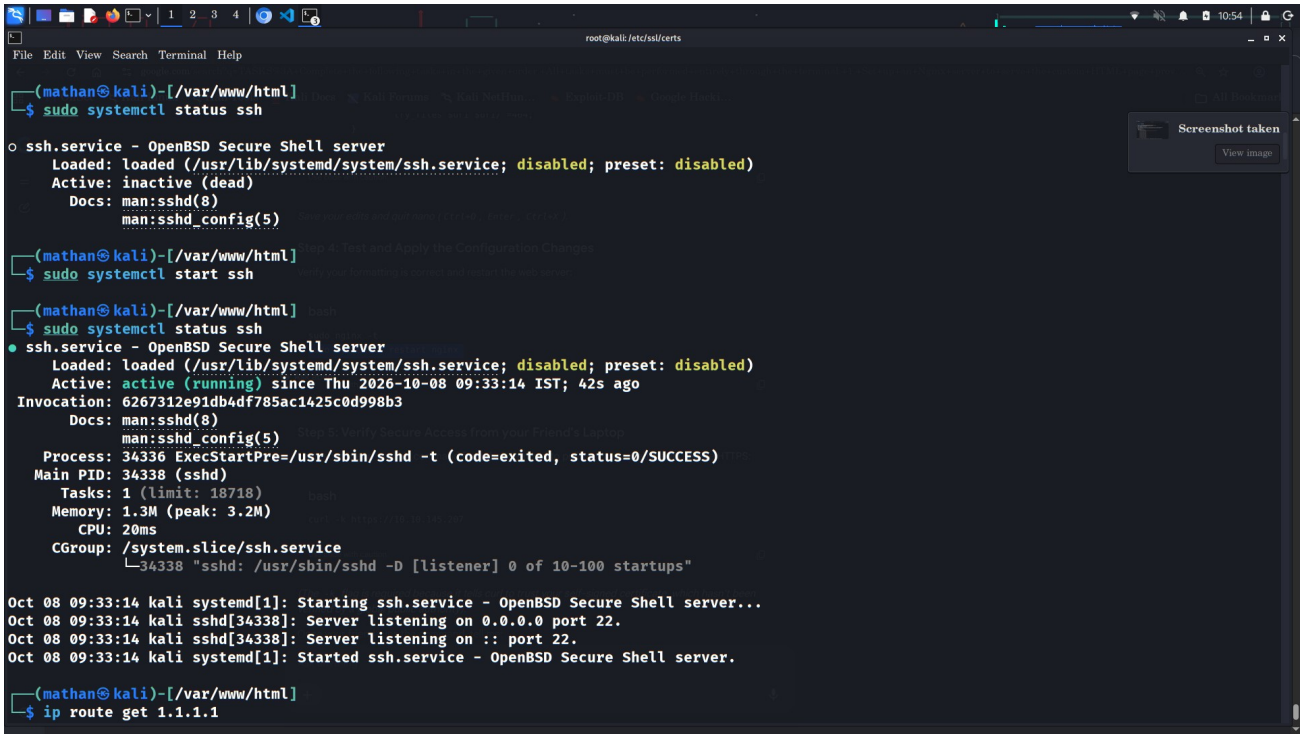
Evidence 3/5 — Screenshot_2026-10-08_10_53_59.png



Evidence 4/5 — Screenshot_2026-10-08_10_54_02.png



Evidence 5/5 — Screenshot_2026-10-08_10_54_09.png


root@kali: /etc/ssl/certs
File Edit View Search Terminal Help
┌──(mathan㉿kali)-[/var/www/html]
└─$ sudo systemctl status ssh
○ ssh.service - OpenBSD Secure Shell server
Loaded: loaded (/usr/lib/systemd/system/ssh.service; disabled; preset: disabled)
Active: inactive (dead)
Docs: man:sshd(8)
man:sshd_config(5)
┌──(mathan㉿kali)-[/var/www/html]
└─$ sudo systemctl start ssh
┌──(mathan㉿kali)-[/var/www/html]
└─$ sudo systemctl status ssh
● ssh.service - OpenBSD Secure Shell server
Loaded: loaded (/usr/lib/systemd/system/ssh.service; disabled; preset: disabled)
Active: active (running) since Thu 2026-10-08 09:33:14 IST; 42s ago
Invocation: 6267312e91db4df785ac1425c0d998b3
Docs: man:sshd(8)
man:sshd_config(5)
Process: 34336 ExecStartPre=/usr/sbin/sshd -t (code=exited, status=0/SUCCESS)
Main PID: 34338 (sshd)
Tasks: 1 (limit: 18718)
Memory: 1.3M (peak: 3.2M)
CPU: 20ms
CGroup: /system.slice/ssh.service
└─34338 "sshd: /usr/sbin/sshd -D [listener] 0 of 10-100 startups"
Oct 08 09:33:14 kali systemd[1]: Starting ssh.service - OpenBSD Secure Shell server...
Oct 08 09:33:14 kali sshd[34338]: Server listening on 0.0.0.0 port 22.
Oct 08 09:33:14 kali sshd[34338]: Server listening on :: port 22.
Oct 08 09:33:14 kali systemd[1]: Started ssh.service - OpenBSD Secure Shell server.
┌──(mathan㉿kali)-[/var/www/html]
└─$ ip route get 1.1.1.1
Screenshot taken
View image

Task 2 — Access the Hosted Page from Another Computer

Make the Nginx-hosted page reachable from another computer connected to the same network.

The server interface was checked with:

ip -br a

The server address shown in the evidence is:

10.10.145.207/20

The routing table also shows:

10.10.144.1

as the gateway.

SSH was enabled to support remote administration:

sudo systemctl start ssh

sudo systemctl status ssh

The SSH service reported:

Server listening on 0.0.0.0 port 22.

Server listening on :: port 22.

A remote SSH connection to the server address was also captured in the evidence.

The custom page was queried using:

curl http://10.10.145.207

The returned HTML includes the organizer page content, demonstrating that the server is serving the intended file.

The competition specifically requires access from **another computer on the same network**.

The screenshots provide network, HTTP, and remote-session evidence, but the final organizer demonstration should explicitly use a second physical computer. On that second computer run:

curl http://10.10.145.207

or open:

http://10.10.145.207

in a browser.

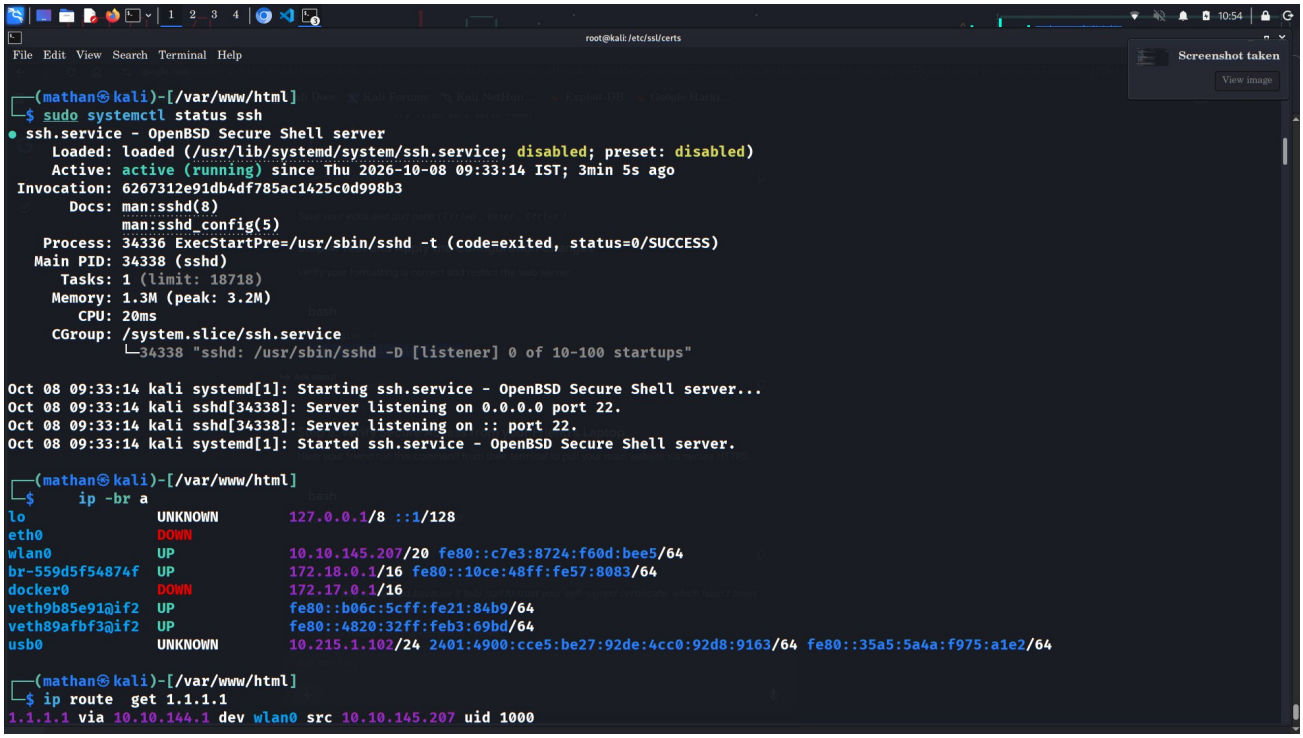
The server is configured on the LAN address `10.10.145.207`, Nginx is reachable, and the custom HTML response is available.

Five screenshots are included in `evidence/task-2-remote-access/`.

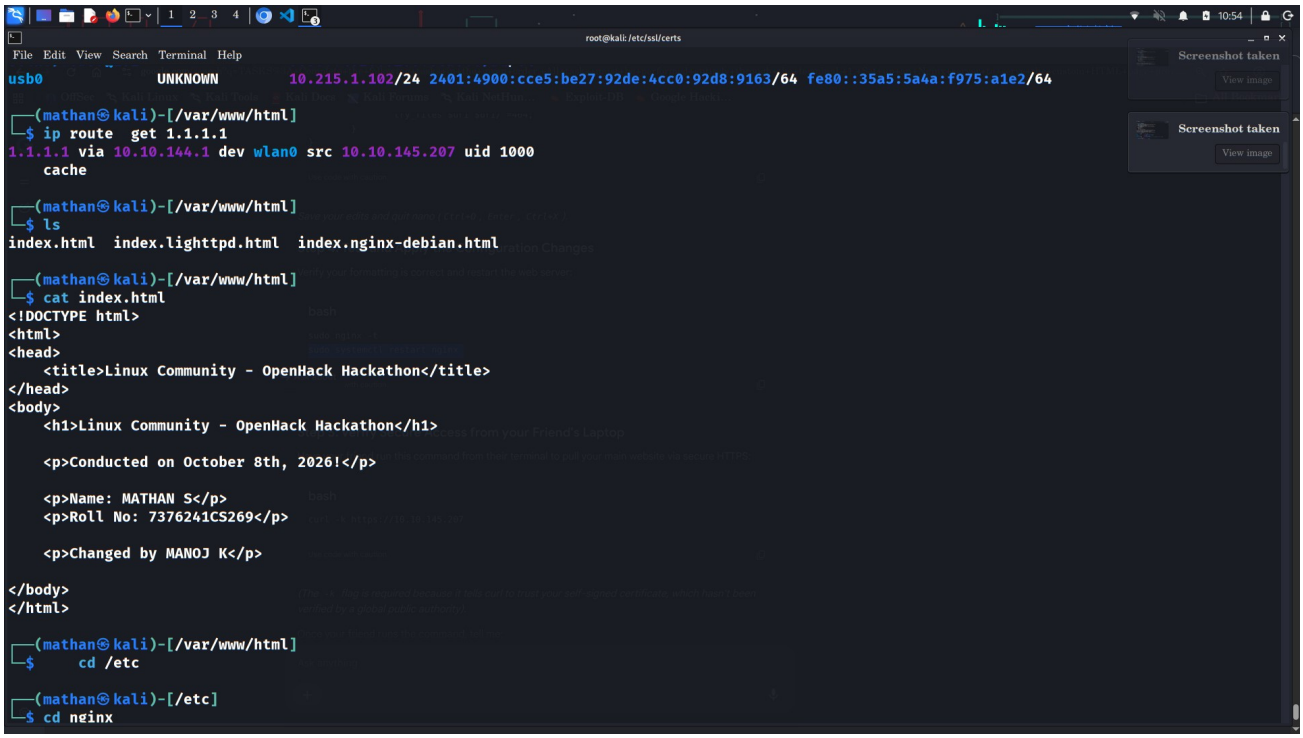
Use a second laptop/PC on the same network and show the address in a browser or terminal.

Terminal Evidence — 5 Screenshots

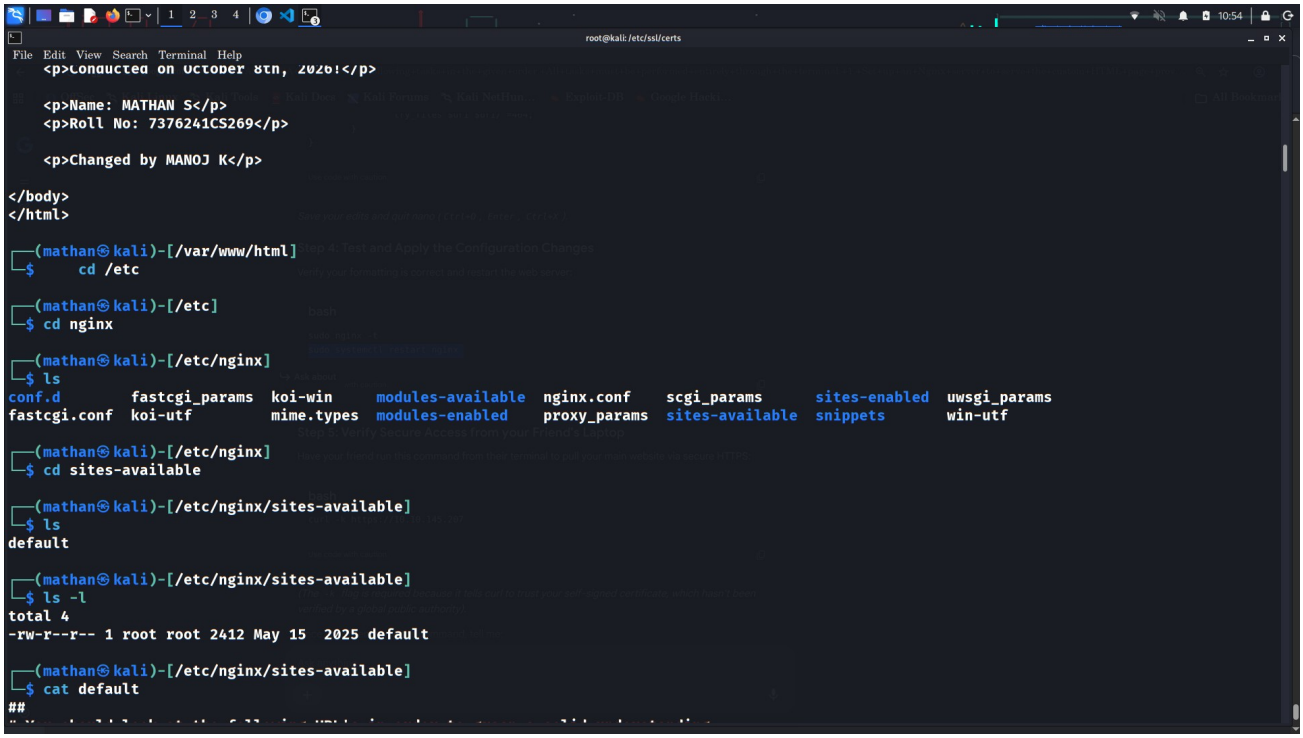
Evidence 1/5 — Screenshot_2026-10-08_10_54_13.png



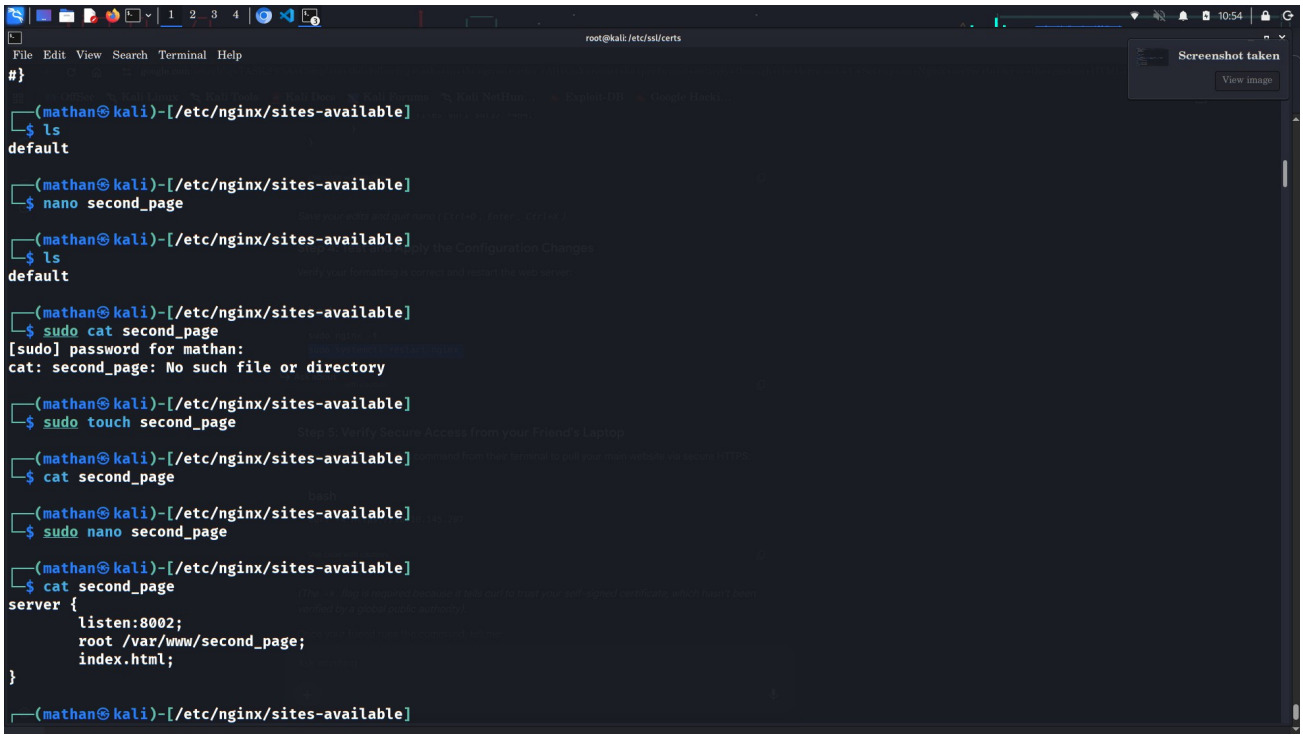
Evidence 2/5 — Screenshot_2026-10-08_10_54_19.png



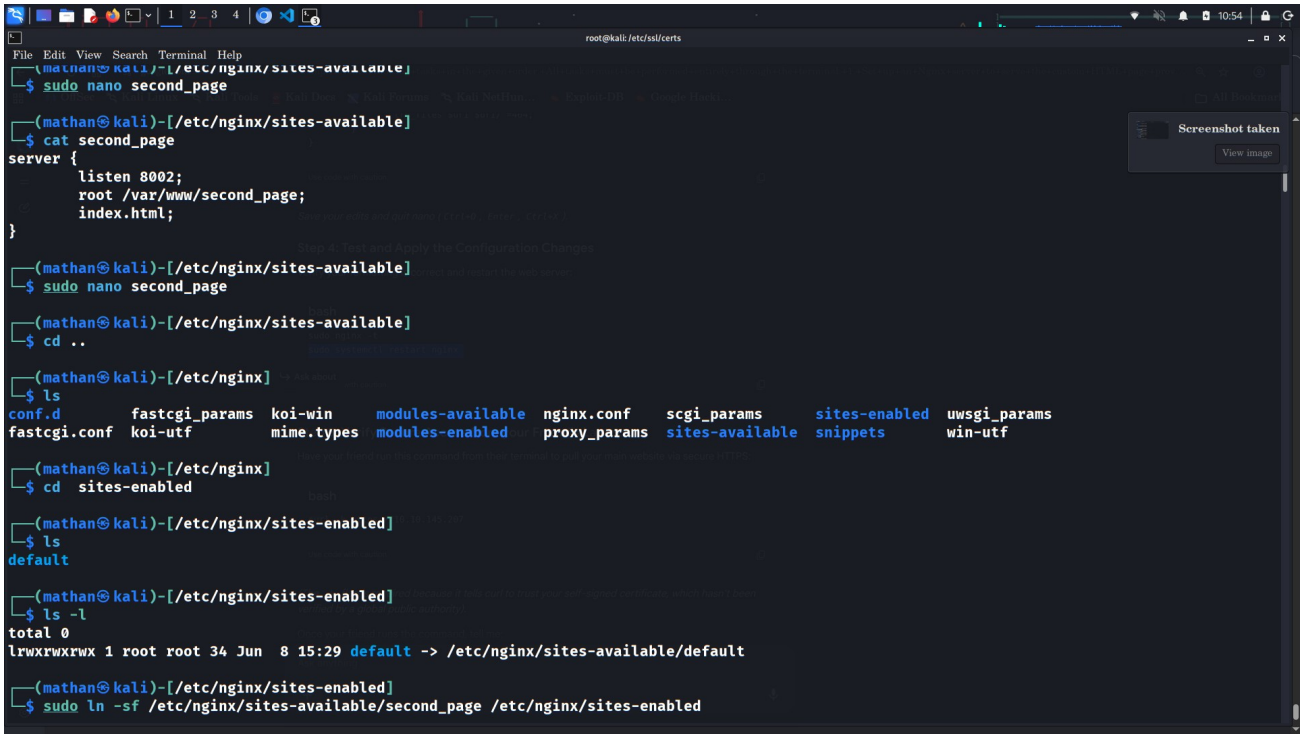
Evidence 3/5 — Screenshot_2026-10-08_10_54_48.png



Evidence 4/5 — Screenshot_2026-10-08_10_54_57.png



Evidence 5/5 — Screenshot_2026-10-08_10_55_00.png


root@kali: /etc/ssl/certs
File Edit View Search Terminal Help
┌──(mathan㉿kali)-[/etc/nginx/sites-available]
└─$ sudo nano second_page
┌──(mathan㉿kali)-[/etc/nginx/sites-available]
└─$ cat second_page
server {
listen 8002;
root /var/www/second_page;
index.html;
}
┌──(mathan㉿kali)-[/etc/nginx/sites-available]
└─$ sudo nano second_page
┌──(mathan㉿kali)-[/etc/nginx/sites-available]
└─$ cd ..
┌──(mathan㉿kali)-[/etc/nginx]
└─$ ls
conf.d fastcgi_params koi-win modules-available nginx.conf scgi_params sites-enabled uwsgi_params
fastcgi.conf koi-utf mime.types modules-enabled proxy_params sites-available snippets win-utf
┌──(mathan㉿kali)-[/etc/nginx]
└─$ cd sites-enabled
┌──(mathan㉿kali)-[/etc/nginx/sites-enabled]
└─$ ls
default
┌──(mathan㉿kali)-[/etc/nginx/sites-enabled]
└─$ ls -l
total 0
lrwxrwxrwx 1 root root 34 Jun 8 15:29 default -> /etc/nginx/sites-available/default
┌──(mathan㉿kali)-[/etc/nginx/sites-enabled]
└─$ sudo ln -sf /etc/nginx/sites-available/second_page /etc/nginx/sites-enabled
Screenshot taken
View image

Task 3 — Modify the HTML Page Remotely and Verify the Change

Modify the hosted HTML page from another computer and demonstrate that Nginx immediately serves the changed content.

The evidence includes an SSH service configured to listen on port 22 and a remote login session to:

10.10.145.207

A remote client can therefore administer the server without physically editing the server's console.

From the second computer, use the permitted remote administration method and edit:

/var/www/html/index.html

For example, with SSH:

ssh mathan@10.10.145.207

Then edit:

sudo nano /var/www/html/index.html

Change a visible value such as the heading or a paragraph.

Save the file and exit.

Nginx does not need to be restarted for a static HTML file change.

Verify from the second computer:

curl http://10.10.145.207

The changed text should appear in the response.

A browser can also be refreshed at:

http://10.10.145.207

The supplied screenshots document the SSH setup, remote session, HTML content, and command history. They do not capture a clean before/after pair of the same HTML element from a clearly identified second physical computer.

Therefore, for the competition's final proof, capture these two additional terminal states if permitted:

1. Before modification — show the original HTML response.

2. After modification — show the changed HTML response from the second computer.

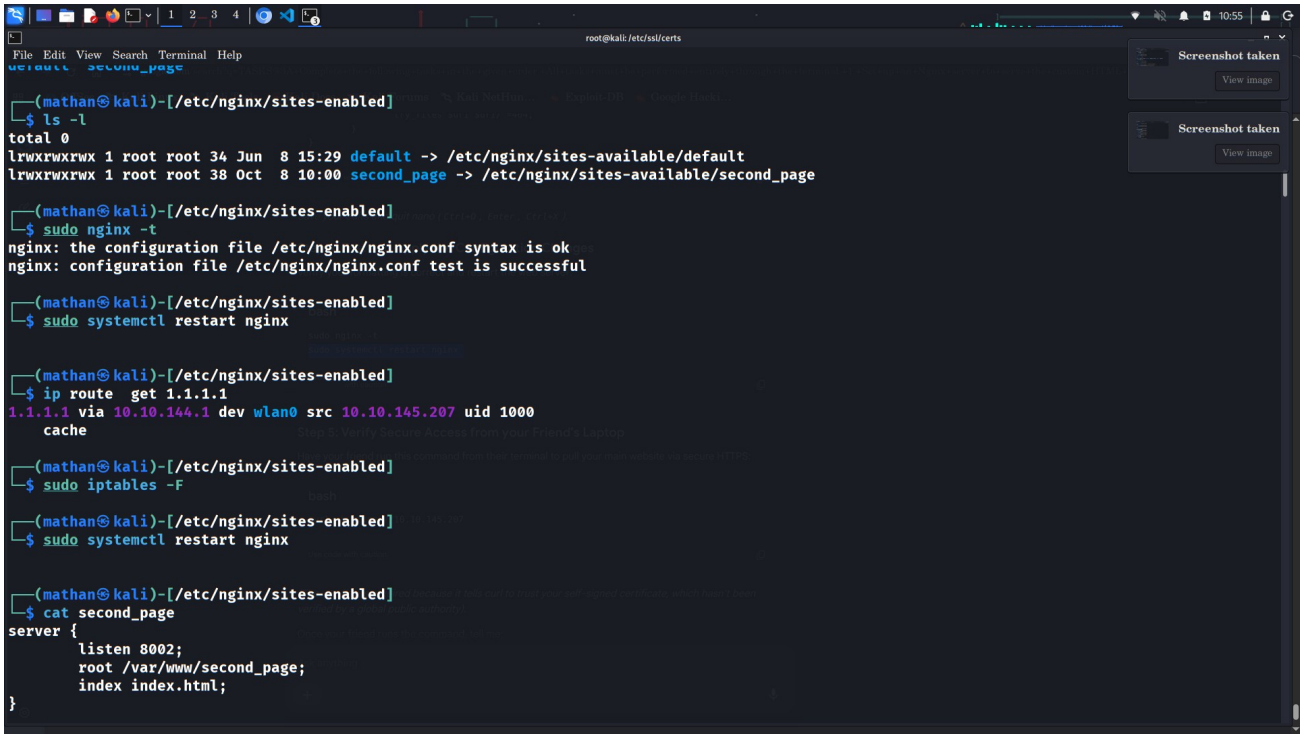
This makes the Task 3 evidence unambiguous.

The architecture supports remote modification because the page is a normal static file served by Nginx. A file change is reflected on the next HTTP request without restarting Nginx.

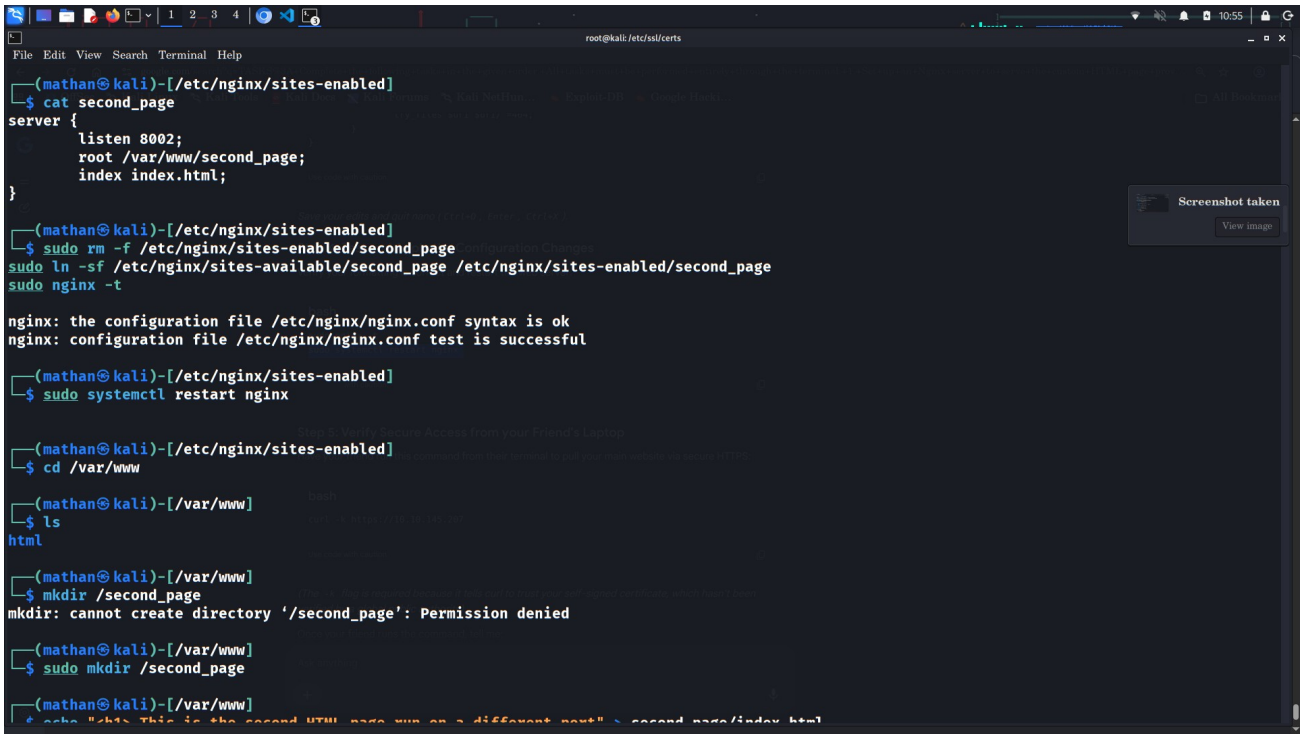
Five screenshots are included in `evidence/task-3-remote-modification/`.

Terminal Evidence — 5 Screenshots

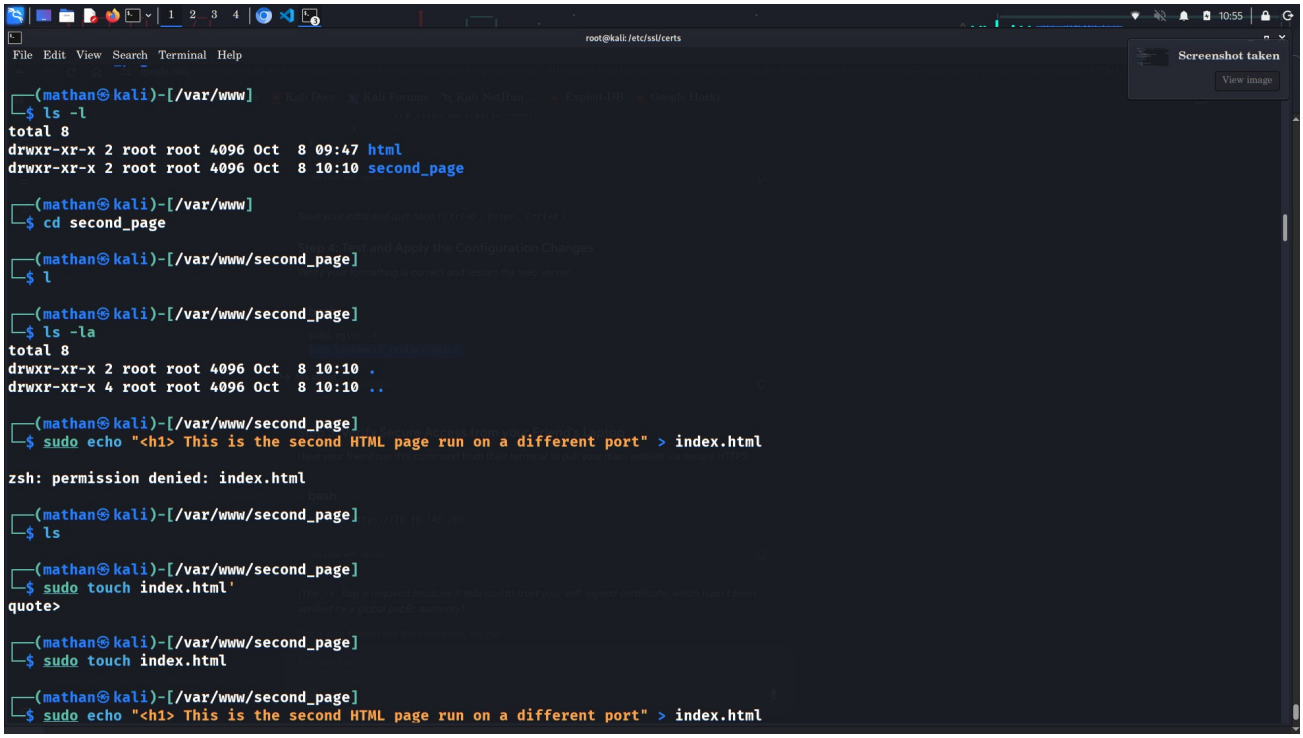
Evidence 1/5 — Screenshot_2026-10-08_10_55_04.png



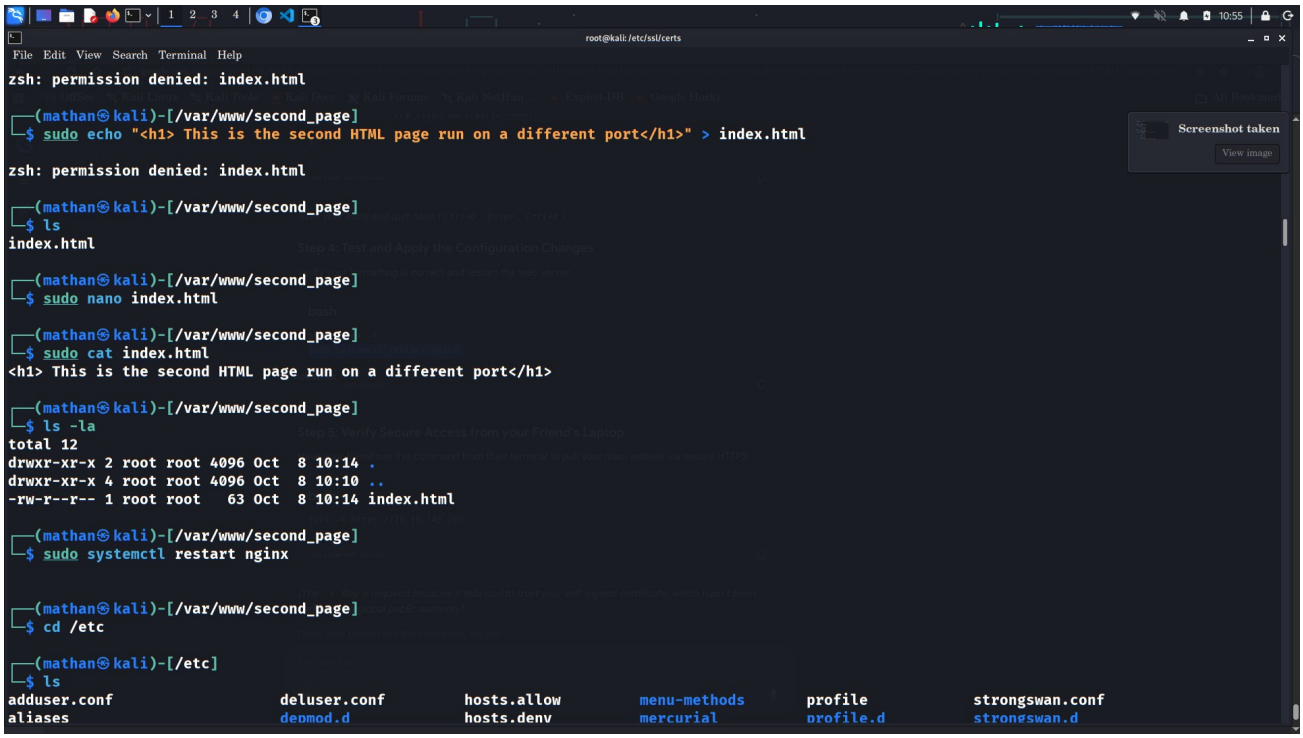
Evidence 2/5 — Screenshot_2026-10-08_10_55_10.png



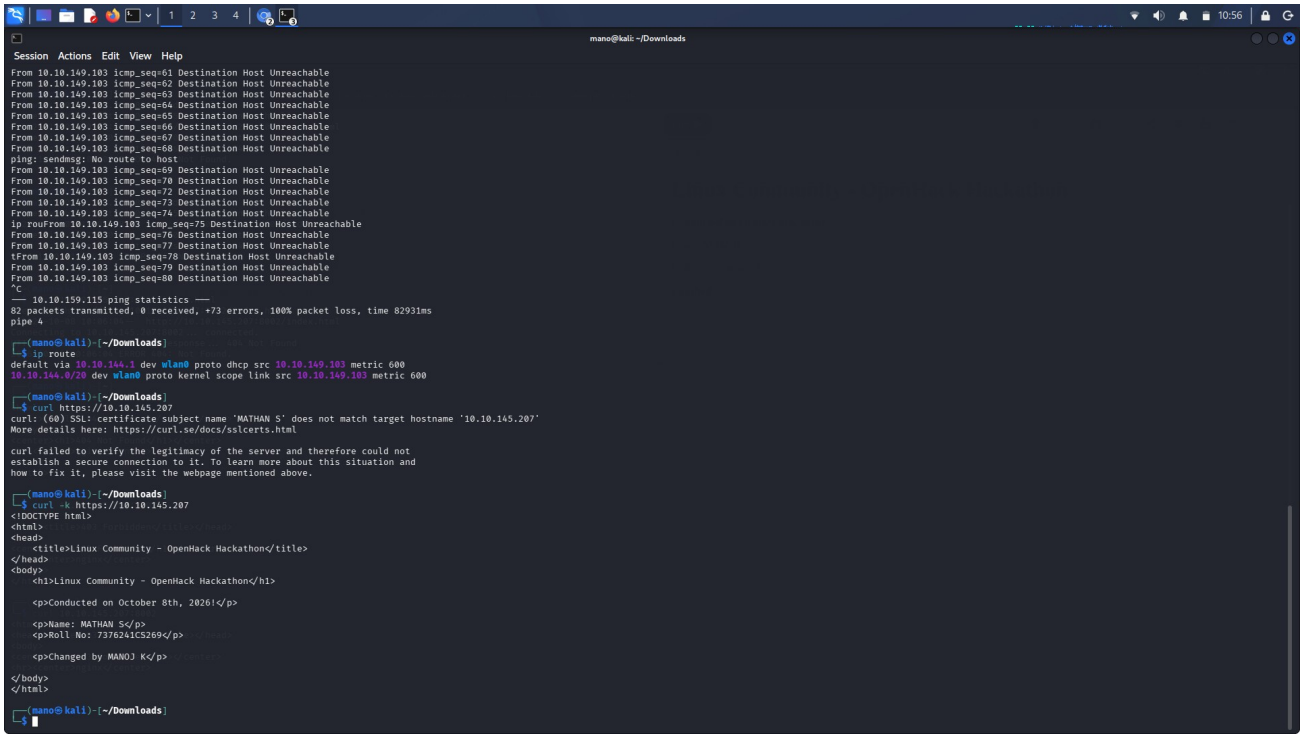
Evidence 3/5 — Screenshot_2026-10-08_10_55_18.png



Evidence 4/5 — Screenshot_2026-10-08_10_55_22.png



Evidence 5/5 — Screenshot_2026-10-08_10_56_24.png


mano@kali: ~/Downloads
Session Actions Edit View Help
From 10.10.149.103 icmp_seq=61 Destination Host Unreachable
From 10.10.149.103 icmp_seq=62 Destination Host Unreachable
From 10.10.149.103 icmp_seq=63 Destination Host Unreachable
From 10.10.149.103 icmp_seq=64 Destination Host Unreachable
From 10.10.149.103 icmp_seq=65 Destination Host Unreachable
From 10.10.149.103 icmp_seq=66 Destination Host Unreachable
From 10.10.149.103 icmp_seq=67 Destination Host Unreachable
From 10.10.149.103 icmp_seq=68 Destination Host Unreachable
ping: sendmsg: No route to host
From 10.10.149.103 icmp_seq=69 Destination Host Unreachable
From 10.10.149.103 icmp_seq=70 Destination Host Unreachable
From 10.10.149.103 icmp_seq=72 Destination Host Unreachable
From 10.10.149.103 icmp_seq=73 Destination Host Unreachable
From 10.10.149.103 icmp_seq=74 Destination Host Unreachable
ip rouFrom 10.10.149.103 icmp_seq=75 Destination Host Unreachable
From 10.10.149.103 icmp_seq=76 Destination Host Unreachable
From 10.10.149.103 icmp_seq=77 Destination Host Unreachable
tFrom 10.10.149.103 icmp_seq=78 Destination Host Unreachable
From 10.10.149.103 icmp_seq=79 Destination Host Unreachable
From 10.10.149.103 icmp_seq=80 Destination Host Unreachable
^C
— 10.10.159.115 ping statistics —
82 packets transmitted, 0 received, +73 errors, 100% packet loss, time 82931ms
pipe 4
┌──(mano㉿kali)-[~/Downloads]
└─$ ip route
default via 10.10.144.1 dev wlan0 proto dhcp src 10.10.149.103 metric 600
10.10.144.0/20 dev wlan0 proto kernel scope link src 10.10.149.103 metric 600
┌──(mano㉿kali)-[~/Downloads]
└─$ curl https://10.10.145.207
curl: (60) SSL: certificate subject name 'MATHAN S' does not match target hostname '10.10.145.207'
More details here: https://curl.se/docs/sslcerts.html
curl failed to verify the legitimacy of the server and therefore could not
establish a secure connection to it. To learn more about this situation and
how to fix it, please visit the webpage mentioned above.
┌──(mano㉿kali)-[~/Downloads]
└─$ curl -k https://10.10.145.207
<!DOCTYPE html>
<html>
<head>
<title>Linux Community - OpenHack Hackathon</title>
</head>
<body>
<h1>Linux Community - OpenHack Hackathon</h1>
Conducted on October 8th, 2026!
Name: MATHAN S
Roll No: 7376241CS269
Changed by MANOJ K
</body>
</html>
┌──(mano㉿kali)-[~/Downloads]
└─$

Task 4 — Serve a Different HTML Page on a Separate Port

Create a second HTML page and serve it from Nginx on a different port.

The second document root is:

/var/www/second_page

The HTML file contains:

<h1>This is the second HTML page run on a different port</h1>

A separate site configuration was created at:

/etc/nginx/sites-available/second_page

Configuration:

server {

listen 8002;

root /var/www/second_page;

index index.html;

}

The site was enabled with:

sudo ln -sf /etc/nginx/sites-available/second_page \

/etc/nginx/sites-enabled/second_page

sudo nginx -t

The evidence shows:

syntax is ok

test is successful

Then Nginx was restarted:

sudo systemctl restart nginx

The final verification was performed with:

curl 10.10.145.207:8002

The successful response was:

<h1>This is the second HTML page run on a different port</h1>

The terminal also shows earlier `404 Not Found` responses while the second page was being prepared. This was resolved after creating the correct document root and `index.html`.

Task 4 was successfully implemented: the second HTML page is served independently through Nginx on TCP port `8002`.

Five screenshots are included in `evidence/task-4-second-port/`.

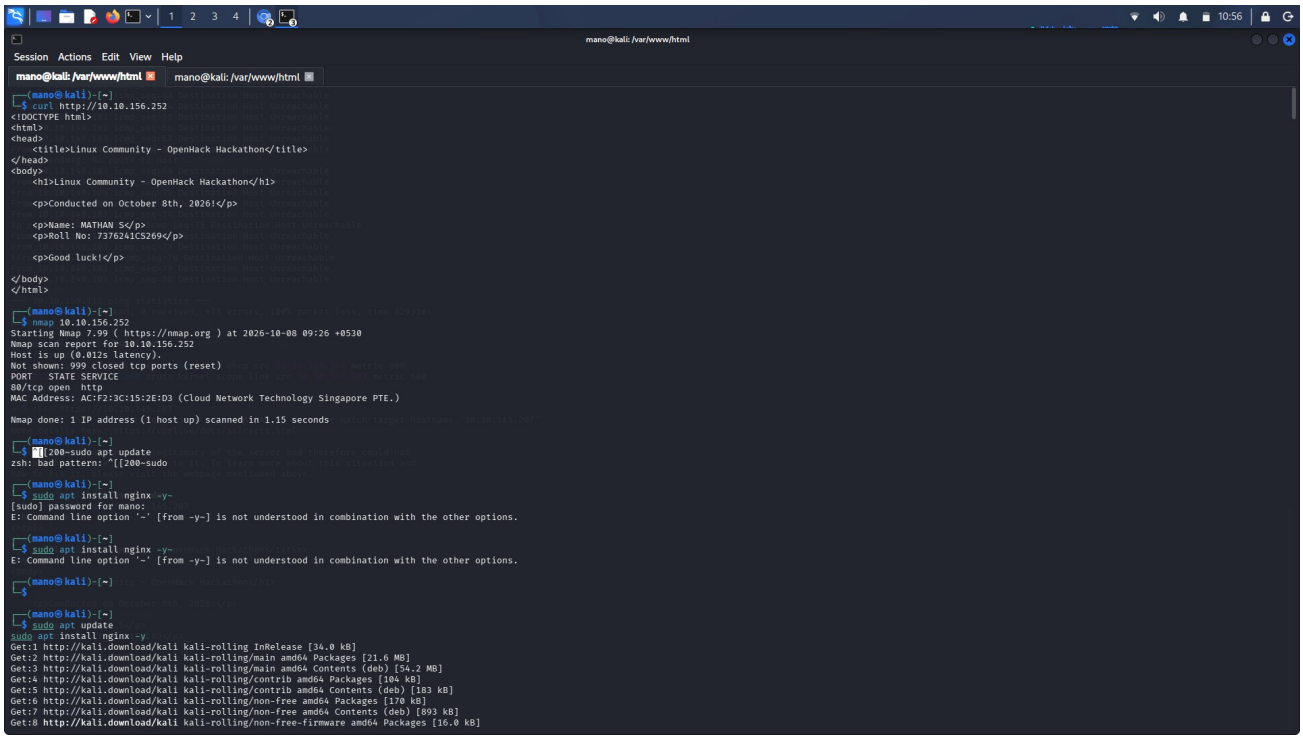
Run:

curl http://10.10.145.207:8002

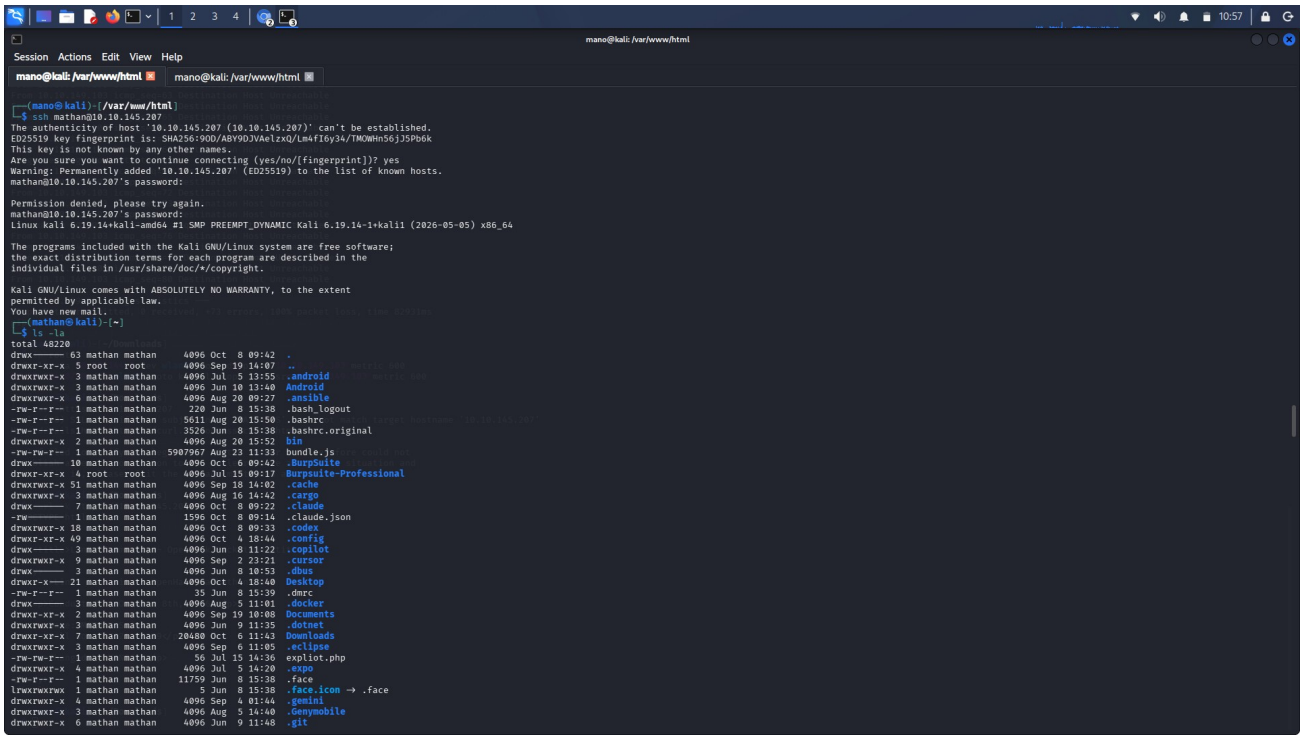
and show the returned second-page heading.

Terminal Evidence — 5 Screenshots

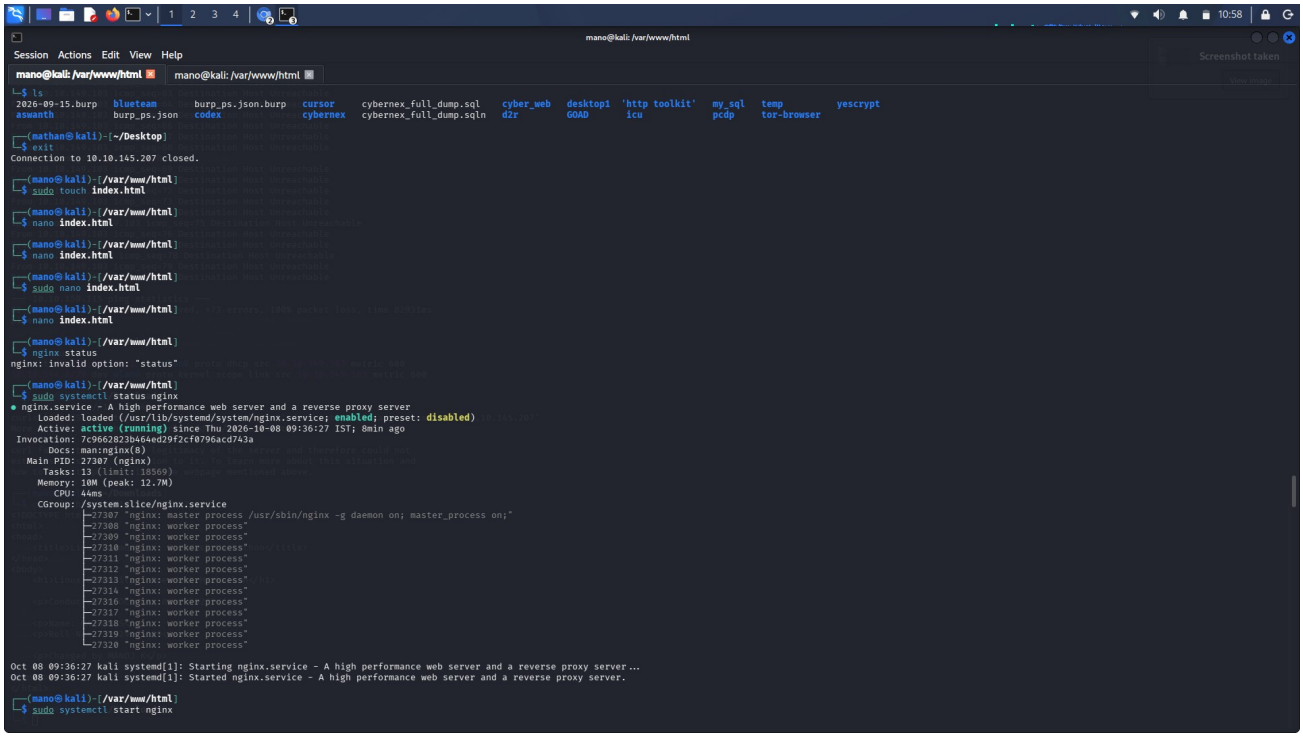
Evidence 1/5 — Screenshot_2026-10-08_10_56_46.png



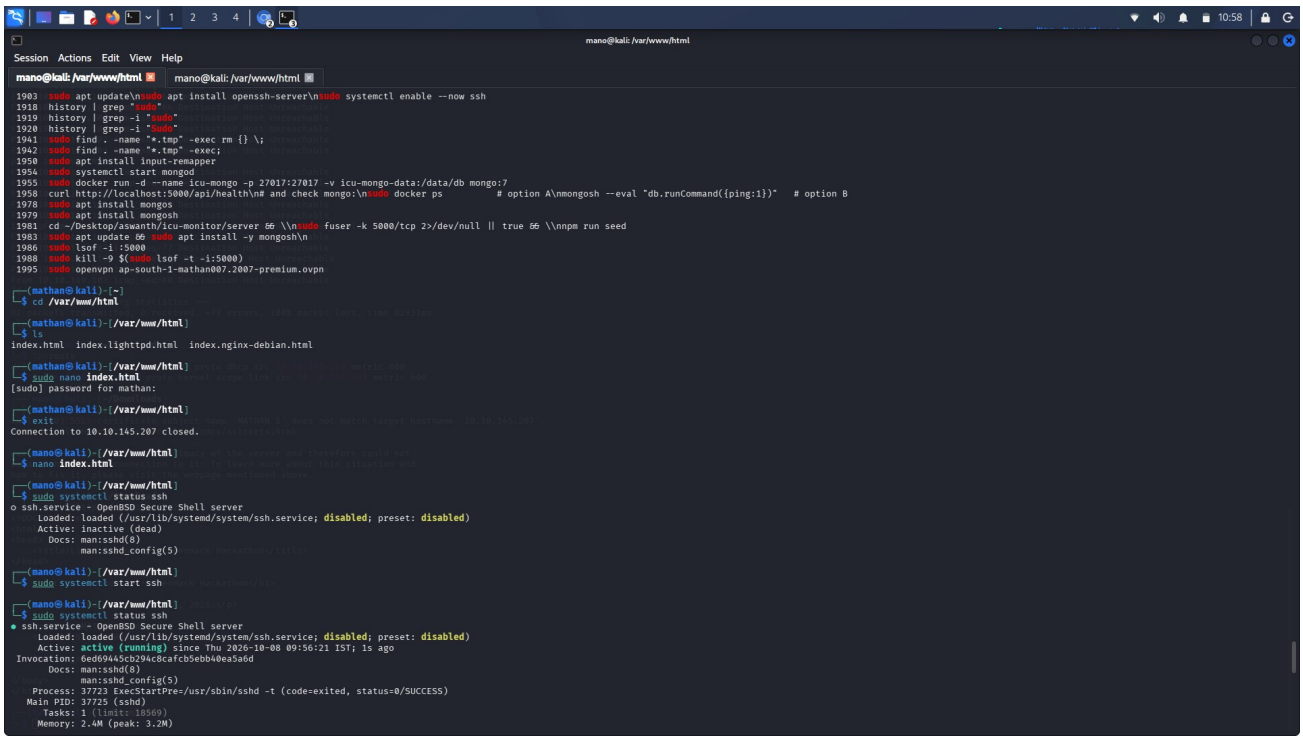
Evidence 2/5 — Screenshot_2026-10-08_10_57_52.png



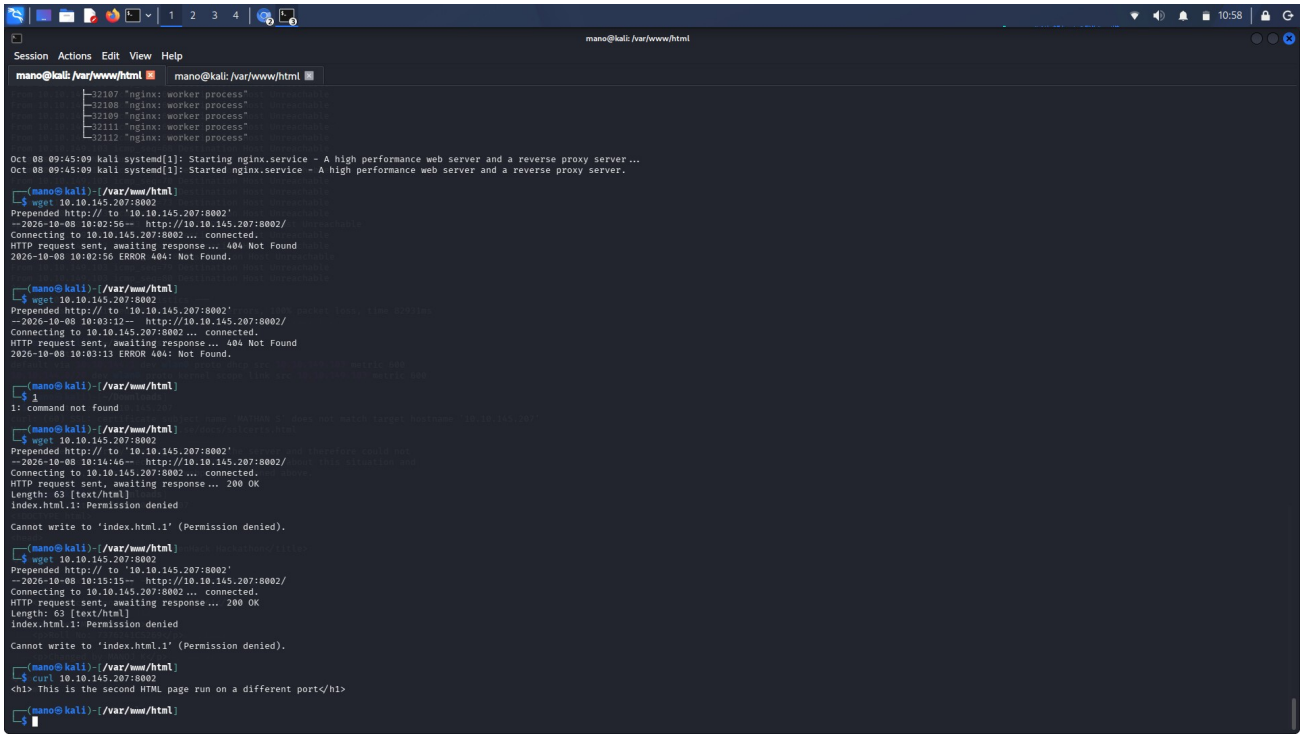
Evidence 3/5 — Screenshot_2026-10-08_10_58_02.png



Evidence 4/5 — Screenshot_2026-10-08_10_58_23.png



Evidence 5/5 — Screenshot_2026-10-08_10_58_37.png


mano@kali: /var/www/html
Session Actions Edit View Help
mano@kali: /var/www/html
mano@kali: /var/www/html
├─32107 "nginx: worker process"
├─32108 "nginx: worker process"
├─32109 "nginx: worker process"
├─32111 "nginx: worker process"
└─32112 "nginx: worker process"
Oct 08 09:45:09 kali systemd[1]: Starting nginx.service - A high performance web server and a reverse proxy server...
Oct 08 09:45:09 kali systemd[1]: Started nginx.service - A high performance web server and a reverse proxy server.
┌──(mano㉿kali)-[/var/www/html]
└─$ wget 10.10.145.207:8002
Prepended http:// to '10.10.145.207:8002'
--2026-10-08 10:02:56-- http://10.10.145.207:8002/
Connecting to 10.10.145.207:8002... connected.
HTTP request sent, awaiting response... 404 Not Found
2026-10-08 10:02:56 ERROR 404: Not Found.
┌──(mano㉿kali)-[/var/www/html]
└─$ wget 10.10.145.207:8002
Prepended http:// to '10.10.145.207:8002'
--2026-10-08 10:03:12-- http://10.10.145.207:8002/
Connecting to 10.10.145.207:8002... connected.
HTTP request sent, awaiting response... 404 Not Found
2026-10-08 10:03:13 ERROR 404: Not Found.
┌──(mano㉿kali)-[/var/www/html]
└─$ 1
1: command not found
┌──(mano㉿kali)-[/var/www/html]
└─$ wget 10.10.145.207:8002
Prepended http:// to '10.10.145.207:8002'
--2026-10-08 10:14:46-- http://10.10.145.207:8002/
Connecting to 10.10.145.207:8002... connected.
HTTP request sent, awaiting response... 200 OK
Length: 63 [text/html]
index.html.1: Permission denied
Cannot write to 'index.html.1' (Permission denied).
┌──(mano㉿kali)-[/var/www/html]
└─$ wget 10.10.145.207:8002
Prepended http:// to '10.10.145.207:8002'
--2026-10-08 10:15:15-- http://10.10.145.207:8002/
Connecting to 10.10.145.207:8002... connected.
HTTP request sent, awaiting response... 200 OK
Length: 63 [text/html]
index.html.1: Permission denied
Cannot write to 'index.html.1' (Permission denied).
┌──(mano㉿kali)-[/var/www/html]
└─$ curl 10.10.145.207:8002
<h1> This is the second HTML page run on a different port</h1>
┌──(mano㉿kali)-[/var/www/html]
└─$

Task 5 — Convert the Hosted Page to HTTPS

Serve the custom HTML page through HTTPS using Nginx.

A self-signed RSA certificate was generated with OpenSSL:

sudo openssl req -x509 -nodes -days 365 \

-newkey rsa:2048 \

-keyout /etc/ssl/private/nginx-selfsigned.key \

-out /etc/ssl/certs/nginx-selfsigned.crt

Certificate subject information was entered during the OpenSSL prompts.

The Nginx default server configuration was edited to enable SSL on port `443`.

The relevant configuration concept is:

listen 443 ssl default_server;

listen [::]:443 ssl default_server;

and the server uses the generated certificate and private key.

sudo nginx -t

The evidence shows:

nginx: the configuration file /etc/nginx/nginx.conf syntax is ok

nginx: configuration file /etc/nginx/nginx.conf test is successful

Nginx was then restarted:

sudo systemctl restart nginx

Because the certificate is self-signed, normal certificate validation will reject it as untrusted.

For local verification:

curl -k https://10.10.145.207

The `-k` option tells curl to continue despite the self-signed certificate.

The HTTPS request returned the custom HTML page, confirming that the page was being served over TLS.

This certificate is appropriate for a local hackathon/lab demonstration. It is **not a trusted production certificate**.

For a production deployment, use a certificate issued by a trusted CA and configure a proper hostname.

HTTPS was configured and the custom page was successfully returned through the HTTPS endpoint.

Five screenshots are included in `evidence/task-5-https/`.

Run:

curl -k https://10.10.145.207

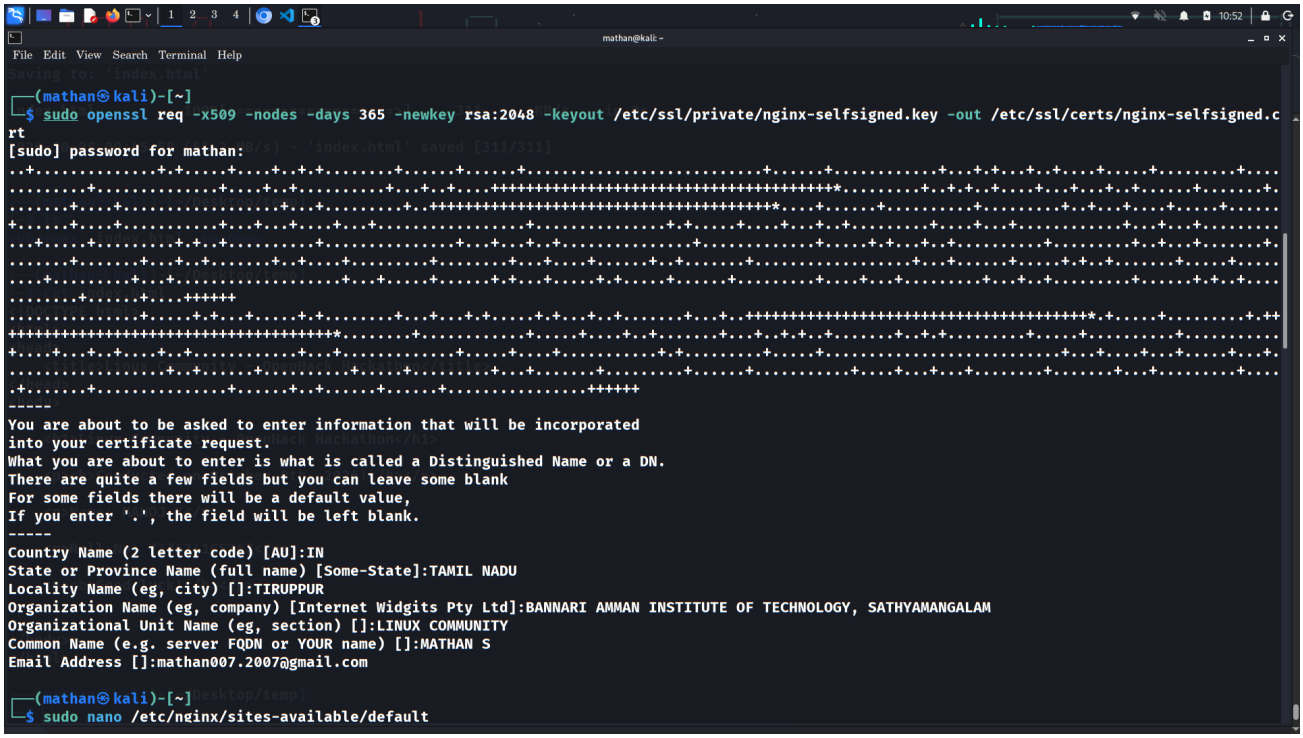
Then open:

https://10.10.145.207

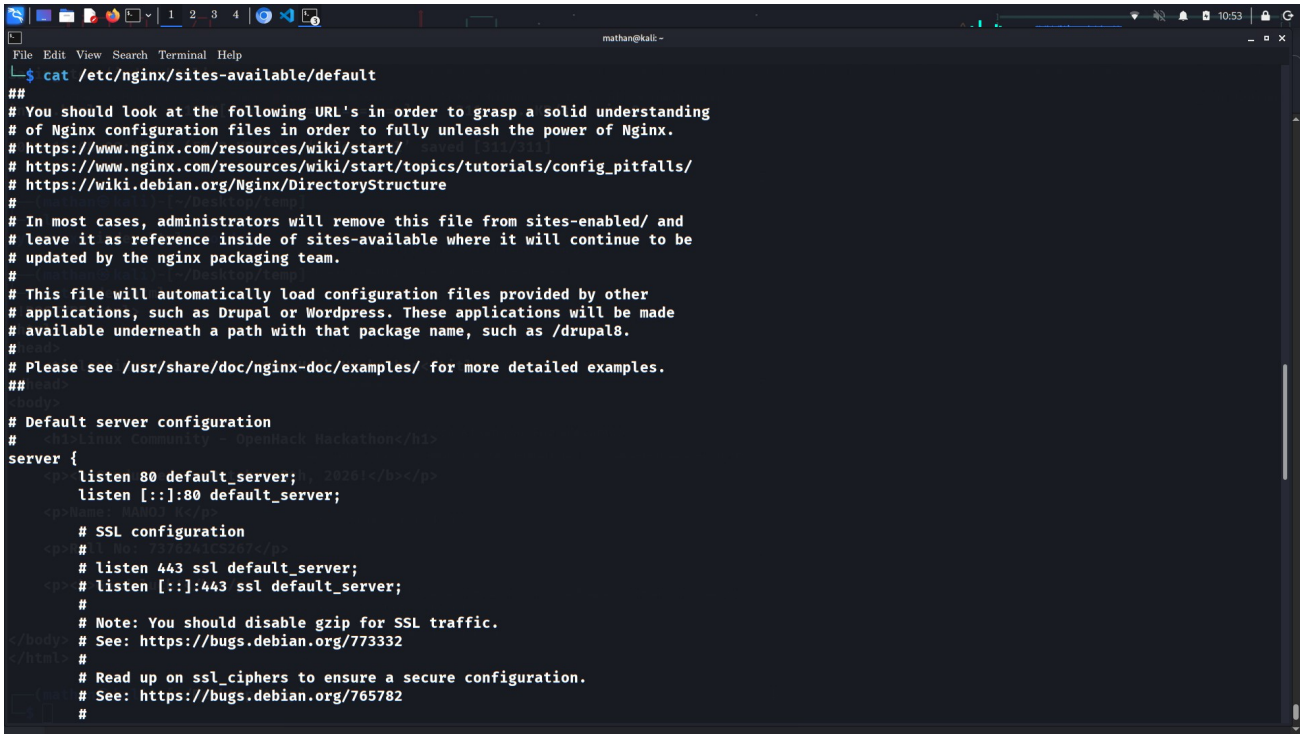
in a browser and explain that the browser warning is expected because the certificate is self-signed.

Terminal Evidence — 5 Screenshots

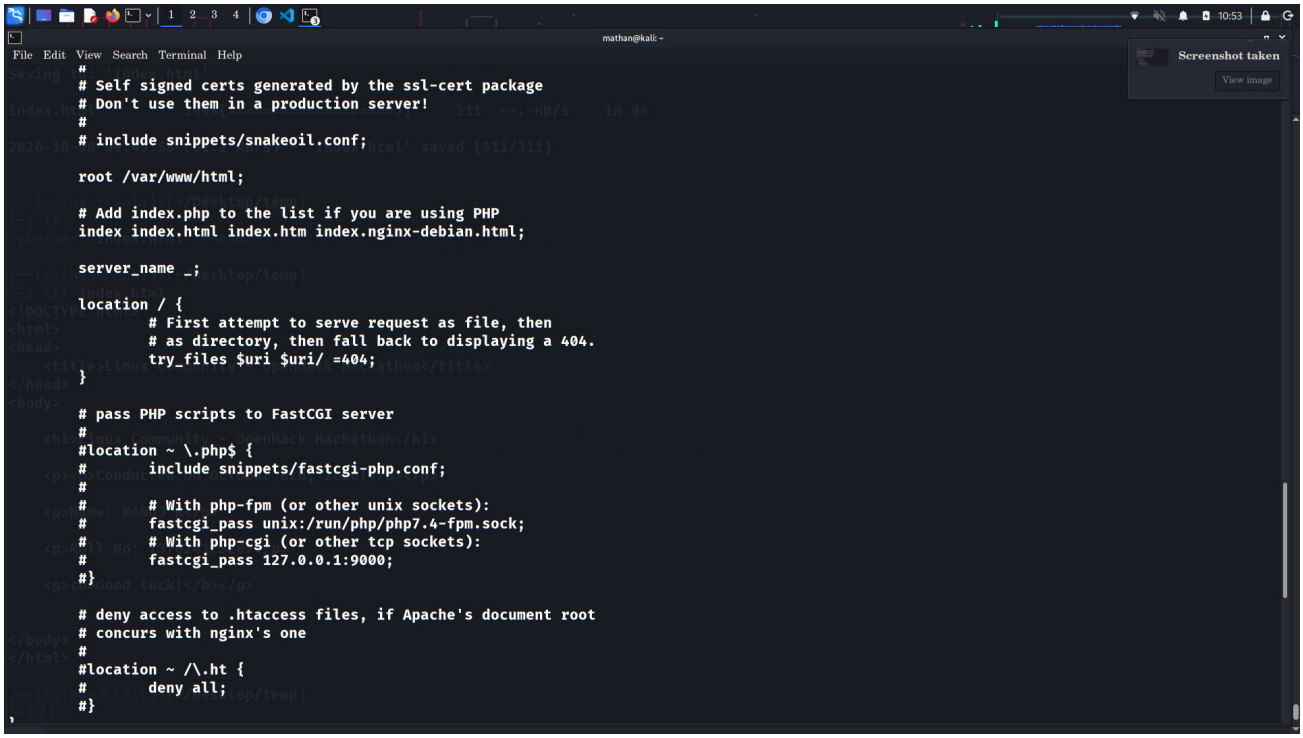
Evidence 1/5 — Screenshot_2026-10-08_10_52_41.png



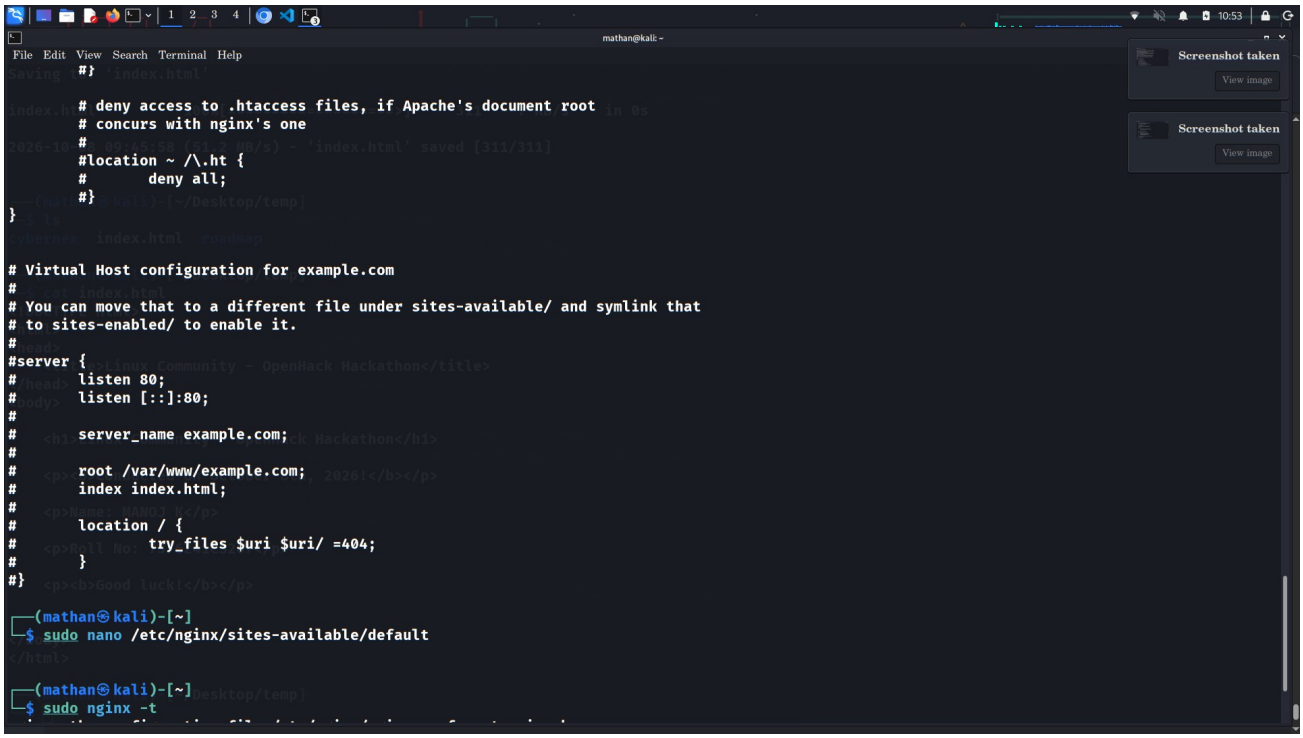
Evidence 2/5 — Screenshot_2026-10-08_10_53_01.png



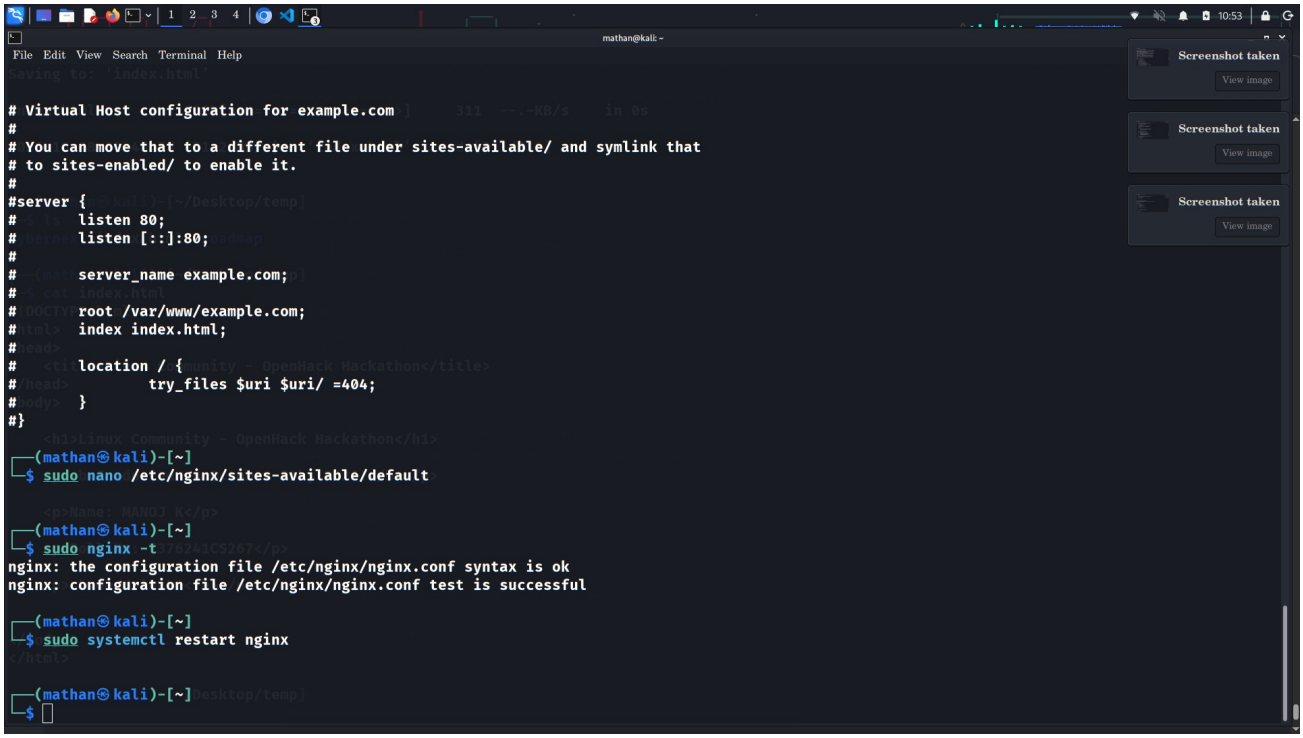
Evidence 3/5 — Screenshot_2026-10-08_10_53_03.png



Evidence 4/5 — Screenshot_2026-10-08_10_53_05.png



Evidence 5/5 — Screenshot_2026-10-08_10_53_08.png


mathan@kali: ~
File Edit View Search Terminal Help
Virtual Host configuration for example.com
#
You can move that to a different file under sites-available/ and symlink that
to sites-enabled/ to enable it.
#
#server {
#	listen 80;
#	listen [::]:80;
#
#	server_name example.com;
#
#	root /var/www/example.com;
#	index index.html;
#
#	location / {
#		try_files $uri $uri/ =404;
#	}
#}
┌──(mathan㉿kali)-[~]
└─$ sudo nano /etc/nginx/sites-available/default
┌──(mathan㉿kali)-[~]
└─$ sudo nginx -t
nginx: the configuration file /etc/nginx/nginx.conf syntax is ok
nginx: configuration file /etc/nginx/nginx.conf test is successful
┌──(mathan㉿kali)-[~]
└─$ sudo systemctl restart nginx
┌──(mathan㉿kali)-[~]
└─$
Screenshot taken
View image
Screenshot taken
View image
Screenshot taken
View image

Final Submission Notes

The screenshots are preserved exactly as supplied. The documentation does not claim that a local curl request proves the separate-computer requirement. For Tasks 2 and 3, the final live demonstration should be made from a second physical computer on the same LAN.

The HTTPS certificate is self-signed. A browser trust warning is therefore expected; successful TLS serving is verified with curl -k.